

Fiche de révision : SMOTE, validation croisée, élagage

Predicting User Sentiment in Software Issue Reports · Yann MASSOUAM, Vénus BAKIKO, Faïçal DIELO

1. SMOTE et le rééchantillonnage (régler le déséquilibre)

Le problème. Nos avis sont très déséquilibrés : $\sim 64\%$ positifs, $\sim 31\%$ négatifs, mais seulement $\sim 4,5\%$ neutres. Le modèle apprend qu'il a presque toujours raison en ignorant le neutre.

Image : une classe de fête avec très peu d'invités « neutres ». Deux façons d'équilibrer la salle :

- **SMOTE** (*Synthetic Minority Over-sampling*) : on **fabrique de nouveaux neutres synthétiques**. On prend un avis neutre, un de ses voisins neutres proches ($k = 5$ voisins), et on crée un point « entre les deux » (interpolation). On ne copie pas bêtement : on invente des exemples plausibles entre des vrais.
- **Sous-échantillonnage** : on **retire des positifs** (la classe majoritaire) jusqu'à rééquilibrer. Simple, mais on jette de l'information.

La règle d'or anti-fuite

Le rééchantillonnage se fait **uniquement sur l'entraînement**, jamais sur le test. On l'a mis *dans* un pipeline `imblearn`, donc à chaque pli de validation croisée, SMOTE ne voit que les données d'entraînement de ce pli. Sinon on évaluerait sur des points inventés $\Rightarrow$ score truqué.

Stratégie	Accuracy	Macro-F1	Rappel neutre
aucune (référence)	0,84	0,56	0,00
SMOTE (créé des neutres synthétiques)	0,73	0,55	0,19
sous-échantillonnage	0,60	0,51	0,47

Ce qu'on en conclut. Le rééchantillonnage **fait exister la classe rare** (rappel neutre $0 \rightarrow 0,19 \rightarrow 0,47$) **au prix d'un peu d'accuracy** : on ne crée pas de performance gratuite, on *redistribue* les erreurs. Le choix dépend de l'objectif : si remonter les neutres est prioritaire, le sous-échantillonnage gagne (rappel 0,47) ; sinon SMOTE est le meilleur compromis.

2. La validation croisée (juger un modèle honnêtement)

Le problème. Si on règle et on juge le modèle sur les *mêmes* données, il a juste « appris par cœur » : le score est trop optimiste. Et un seul découpage train/test peut tomber sur une part « facile » ou « difficile » par chance.

Image : un examen blanc tournant. On coupe l'entraînement en **5 parts (plis)**. À tour de rôle, **4 parts servent à apprendre** et la **5^e sert d'examen**. On recommence 5 fois pour que chaque part passe une fois l'examen, puis on **moyenne les 5 scores**.

	pli 1	pli 2	pli 3	pli 4	pli 5	
tour 1	TEST	train	train	train	train	$\rightarrow \text{score}_1$
tour 2	train	TEST	train	train	train	$\rightarrow \text{score}_2$
tour 3	train	train	TEST	train	train	$\rightarrow \text{score}_3$
tour 4	train	train	train	TEST	train	$\rightarrow \text{score}_4$
tour 5	train	train	train	train	TEST	$\rightarrow \text{score}_5$

score final = $\frac{\text{score}_1 + \dots + \text{score}_5}{5}$

- **Stratifiée** (`StratifiedKFold`) : chaque pli garde les **mêmes proportions** de classes que l'ensemble (crucial vu notre déséquilibre, sinon un pli pourrait n'avoir aucun neutre).
- **Couplée à GridSearchCV** : on teste chaque combinaison d'hyper-paramètres en validation croisée et on garde la meilleure, mesurée sur le **macro-F1** (pas l'accuracy, pour ne pas récompenser un modèle qui ignore le neutre).
- **Graine fixe** (`RANDOM_STATE = 42`) : mêmes plis à chaque exécution $\Rightarrow$ résultats reproductibles.

Pourquoi c'est essentiel

La validation croisée donne une **estimation stable et honnête** de la performance sur des données *jamais vues*, et empêche le **sur-apprentissage** du réglage : on ne choisit pas les hyper-paramètres qui plaisent à un

seul découpage chanceux. Le vrai test final (les 1 185 avis mis de côté) reste, lui, totalement intact.

3. L'élagage de l'arbre de décision (lutter contre le sur-apprentissage)

Le problème. Un arbre de décision laissé libre grandit jusqu'à **mémoriser** l'entraînement (une feuille par cas particulier). Il est parfait sur le train et médiocre sur le test : c'est le **sur-apprentissage**.

Image : tailler un arbre. On coupe les branches inutiles pour garder la forme générale, pas chaque petite brindille. Deux moments pour tailler :

- **Pré-élagage** (pre-pruning) : on **empêche l'arbre de trop pousser** dès le départ (limiter la profondeur, exiger un minimum d'exemples par feuille). On taille *pendant* la croissance.
- **Post-élagage** (post-pruning) : on **laisse l'arbre pousser en entier, puis on coupe** les branches qui n'apportent rien, via le **coût-complexité** (ccp_alpha) : plus ccp_alpha est grand, plus on coupe.

Arbre	Nb de nœuds	Score train	Score test
complet (non élagué)	1 857	0,976	0,759
bien élagué	69	0,811	0,786
sur-élagué	29	0,770	0,770

Comment lire ce tableau (à raconter au jury).

- L'arbre complet : **0,976 sur le train mais 0,759 sur le test** → l'écart énorme = sur-apprentissage manifeste, et 1 857 nœuds illisibles.
- L'arbre élagué : on passe de **1 857 à 69 nœuds**, le score train *baisse* (0,811) mais le score **test monte** (0,786). On a échangé un peu de mémorisation contre de la **généralisation** : c'est exactement le but.
- Trop élaguer (29 nœuds) : le test *redescend* (0,770). L'arbre devient trop simpliste (sous-apprentissage).

Le compromis biais-variance en une image

Trop d'arbre = **variance** (mémorise le bruit). Pas assez = **biais** (trop bête). L'élagage cherche le **point creux au milieu** : assez complexe pour comprendre, assez simple pour généraliser. Notre meilleur arbre (69 nœuds, test 0,786) est ce point d'équilibre, et il est en prime **lisible** (interprétable).

4. Comment on a ajusté les paramètres des modèles

Le principe. On ne devine pas les hyper-paramètres « à la main » : on laisse GridSearchCV essayer une grille de valeurs, chacune évaluée par **validation croisée 5 plis sur le macro-F1**, et il **garde la meilleure combinaison**. Les grilles sont volontairement **compactes** : assez larges pour montrer un vrai réglage, assez petites pour rester rapides sous 5 plis.

Image : régler une recette. On teste quelques dosages (plus / moins de sel), on goûte chaque version à l'aveugle (validation croisée), et on retient le meilleur dosage, pas celui qui nous plaît a priori.

Modèle	Paramètre réglé	Valeurs testées	Ce qu'il contrôle
Naive Bayes	alpha	0,1 / 0,5 / 1,0	lissage de Laplace (mots jamais vus en entraînement)
Decision Tree	max_depth	None / 10 / 20 / 30	pré-élagage : profondeur maximale
	min_samples_leaf	1 / 5 / 10	pré-élagage : taille minimale d'une feuille
	ccp_alpha	0 / 0,001 / 0,01	post-élagage : force du coût-complexité
Random Forest	n_estimators	200	nombre d'arbres
	max_depth	None / 20	profondeur des arbres
	min_samples_leaf	1 / 2	taille minimale d'une feuille
	max_features	sqrt	variables tirées par division
AdaBoost	n_estimators	100 / 200	nombre d'itérations de boosting
	learning_rate	0,5 / 1,0	pas d'apprentissage
Logistic Reg.	C	0,1 / 1,0 / 10,0	force de la régularisation L2

- **Naive Bayes** bascule automatiquement en **GaussianNB** (réglage `var_smoothing`) sur les vecteurs denses (Word2Vec), car **MultinomialNB** exige des comptes positifs.
- **Régularisation (Logistic Regression)** : un *C petit* régularise fort (modèle simple, moins de sur-apprentissage), un *C grand* laisse le modèle coller aux données. Régler *C*, c'est régler ce curseur.
- **Élagage de l'arbre** : les trois paramètres (`max_depth`, `min_samples_leaf`, `ccp_alpha`) sont *dans la grille*, donc le pré- et le post-élagage sont choisis automatiquement (c'est ce qui a donné l'arbre à 69 nœuds de la §3).
- **Early stopping (boosting)** : ce n'est pas une grille mais une propriété du modèle. Le **GradientBoosting** part avec un plafond généreux (`n_estimators=500`) et **s'arrête tout seul** dès que la perte de validation stagne pendant 10 tours (`n_iter_no_change=10`) : il choisit donc lui-même le bon nombre d'arbres.

La phrase à dire au jury

« On n'a pas réglé à la main : **GridSearchCV** teste chaque combinaison en **validation croisée 5 plis sur le macro-F1** et retient la meilleure. La grille couvre exprès le **pré- et post-élagage** (arbres), la **régularisation** (*C* de la régression logistique) et le **boosting** (`n_estimators`, `learning_rate`), et le boosting a en plus un **early stopping** automatique. »

5. Les points subtils qui prouvent la maîtrise

- **SMOTE interpole, il ne duplique pas** : il crée des points *entre* des voisins réels ($k = 5$), ce qui évite le simple sur-poids du même exemple. Et il vit **dans le pipeline**, donc strictement sur le train.
- **Stratification $\neq$ simple découpe** : sans elle, un pli pourrait n'avoir aucun neutre et fausser le macro-F1.
- **On règle sur le macro-F1, pas l'accuracy** : cohérent avec le déséquilibre, pour ne pas primer un modèle qui ignore la classe rare.
- **Élagage = moins de nœuds ET meilleur test** : la preuve qu'on combat le sur-apprentissage, pas qu'on perd de l'information. Le score *train* qui baisse est ici une **bonne** nouvelle.
- **Tout est reproductible** (graine 42) : mêmes plis, mêmes synthétiques, mêmes arbres à chaque exécution.

En une phrase : « SMOTE fait exister la classe rare, la validation croisée 5-plis juge honnêtement sans fuite, et l'élagage troque la mémorisation (1 857 nœuds) contre la généralisation (69 nœuds, test 0,759 $\rightarrow$ 0,786). »